AI-Powered Expense Report Fraud Detection System

Receiptly

CPSC 491 - Fall 2026 Project Proposal
Dr. Kanika Sood

California State University, Fullerton

Team Members: Jim Alvarez, Evan Miller, Syon Chau

Application Project - Software / Web / Mobile Application

Project Phase: CPSC 491 development, evaluation, deployment, and monitoring

1. Abstract

Receiptly is an AI-powered expense fraud detection system designed to help businesses identify suspicious reimbursement claims before approval. During Spring 2026, the project reached a functional MVP with receipt upload, OCR extraction, perceptual-hash duplicate detection, configurable policy validation, a 0-100 fraud risk score, Gemini-generated explanations, PostgreSQL storage, and a React reviewer interface. The Fall 2026 CPSC 491 phase will move the system from a local MVP toward a deployable, multi-business product. Work will focus on improving receipt-reading accuracy, strengthening anomaly and pattern detection, expanding dashboard analytics, adding company-specific customization, creating a mobile receipt-scanning workflow, supporting CSV/PDF exports, performing additional model training and evaluation, deploying the application, and establishing production monitoring.

2. Introduction

Expense reimbursement fraud can take the form of duplicate submissions, altered receipt details, inflated or personal expenses, policy violations, or fabricated/anomalous transactions. Traditional expense review requires finance staff to manually interpret receipt images, compare submissions, and apply company policies consistently. Receiptly addresses this problem by acting as a decision-support system that extracts receipt information, detects suspicious signals, assigns explainable risk scores, and presents evidence to a human reviewer before reimbursement.

The Spring 2026 milestone established the core upload-to-review workflow. The Fall 2026 phase is focused on improving accuracy, product scalability, reviewer usefulness, mobile access, and deployment readiness rather than rebuilding the existing MVP.

3. Scope

The scope of CPSC 491 is to extend the existing Receiptly MVP into a more complete and deployable application. The project will remain focused on receipt-based expense review and fraud-risk decision support. It will not attempt to replace human auditors or provide a fully autonomous reimbursement approval system.

- Improve receipt image extraction accuracy and evaluate a replacement or enhanced extraction library/engine.
- Expand the PostgreSQL-backed architecture for scalability, mobile connectivity, and multi-business support.
- Add company-specific customization for policies, limits, categories/vendors, reviewer settings, and authenticated user access.
- Improve anomaly detection through historical pattern detection and additional model training/tuning.
- Enhance the dashboard with statistics, scores, trend information, and visual graphs.
- Develop a mobile receipt scanning/upload workflow connected to the existing backend.
- Add CSV and/or PDF export for review reports.
- Prepare, deploy, and monitor the application in a production-style environment.

4. Feasibility

The project is feasible within the Fall 2026 semester because the core MVP is already implemented. The backend API, PostgreSQL data model, OCR service, duplicate detector, policy validator, fraud scorer, Gemini explanation feature, and React frontend provide a working foundation for incremental development. The current backend is modular, which allows new extraction, anomaly-detection, analytics, and mobile-facing features to be added without redesigning the complete upload workflow.

The planned technology stack uses tools that are already familiar to the project or have direct integration paths with the current system, including Python/FastAPI, PostgreSQL, React/TypeScript, scikit-learn, Gemini, and image/OCR libraries. Development can be completed and tested locally before hosted deployment. The main

feasibility constraints are model/extraction accuracy across diverse receipts, integration time for the mobile client, cloud-service limits or costs, and the time required for formal evaluation; these are addressed through phased weekly milestones and monitoring.

5. Objectives

- Increase receipt-reading reliability across diverse layouts, image qualities, and phone-captured receipts.
- Support multi-business configuration and user authentication so different companies can maintain independent users, policies, limits, and review settings.
- Improve fraud/anomaly detection by adding historical pattern-based signals beyond single-receipt rules.
- Provide stronger reviewer analytics through dashboard statistics, fraud/risk scores, and visual graphs.
- Enable receipt capture and upload from a mobile application or mobile-focused client.
- Allow reviewers to export review results as CSV and/or PDF reports.
- Complete deployment and establish monitoring for false positives, processing time, database errors, and AI API costs.
- Evaluate system performance using measurable metrics such as precision, recall, F1-score, false positive rate, OCR/extraction accuracy, and processing time.

6. Proposed Activities for CPSC 491

Development will proceed in four phases aligned with the September-December 2026 implementation timeline. Each phase builds on the Spring 2026 MVP and ends with a working increment that can be tested before the next phase.

6.1 September - Foundation and Core Infrastructure

- Change or replace the library/engine used to extract information from receipt images when testing demonstrates a more accurate approach.
- Improve image preprocessing and receipt field extraction for merchant, date/time, total amount, and line items.
- Expand the database architecture to support future scalability and mobile connectivity.
- Establish baseline extraction metrics so accuracy improvements can be measured.

6.2 October - Feature Enhancements and Dashboard

- Implement multi-business customization, company-specific policy configuration, and user authentication.
- Improve anomaly detection through pattern detection across historical submissions.
- Enhance the dashboard with summary statistics, scores, trend information, and visual graphs.

6.3 November - Additions and Development

- Develop the mobile application or mobile-focused receipt scanning/upload client.
- Implement CSV and/or PDF export functionality for review reports.
- Execute additional model training/tuning and test performance on diverse receipt examples.
- Prepare application configuration and services for deployment.

6.4 December - Deployment and Monitoring

- Complete project deployment and end-to-end validation.
- Establish monitoring for false positive rate, processing time, database errors, and AI API costs.
- Complete final evaluation, documentation, and project handoff materials.

7. Project Plan - Weekly Tasks and Tentative Deadlines

The following weekly plan breaks the approved September-December timeline into tentative CPSC 491 development checkpoints. Dates may shift slightly based on testing results and course deadlines, but the sequence of work will remain the same.

Week / Dates	Primary Tasks / Activities	Tentative Deadline	Expected Checkpoint	Primary Owner(s)
Week 1 Sep. 1-6	Audit current OCR/extraction pipeline; compare candidate replacement libraries/engines; define baseline receipt set.	Sep. 6	Baseline and candidate list	Jim
Week 2 Sep. 7-13	Prototype replacement/enhanced extraction path; compare merchant/date/amount/item accuracy against current Tesseract workflow.	Sep. 13	Extraction prototype	Jim
Week 3 Sep. 14-20	Integrate chosen extraction changes; improve preprocessing and low-confidence handling.	Sep. 20	Improved extraction integrated	Jim
Week 4 Sep. 21-27	Expand database/schema for scalability, company ownership, and mobile-facing data needs; run regression tests.	Sep. 27	Scalable DB changes implemented	Syon
Week 5 Sep. 28- Oct. 4	Finalize September infrastructure work; document extraction accuracy changes and database migrations.	Oct. 4	Foundation phase complete	Jim & Syon
Week 6 Oct. 5-11	Implement multi-business configuration, company-specific policies, user authentication, and business-level data separation logic.	Oct. 11	Multi-business support	Syon
Week 7 Oct. 12-18	Implement initial historical pattern/anomaly features and connect them to the fraud-scoring pipeline.	Oct. 18	Pattern detector integrated	Evan
Week 8 Oct. 19-25	Tune anomaly/pattern thresholds; add dashboard statistics and fraud/risk summary views. Dashboard work is shared between Evan and Jim.	Oct. 25	Dashboard metrics + tuning	Evan & Jim
Week 9 Oct. 26- Nov. 1	Add dashboard graphs/trends, refine analytics, and perform October integration testing.	Nov. 1	October feature phase complete	Evan & Jim
Week 10 Nov. 2-8	Develop mobile receipt capture/upload workflow and connect it to existing backend endpoints.	Nov. 8	Mobile upload prototype	Syon
Week 11 Nov. 9-15	Complete mobile review/upload integration; implement CSV/PDF report export, with Jim leading report extraction/export work.	Nov. 15	Mobile + export features	Syon & Jim
Week 12 Nov. 16-22	Perform additional fraud-model training/tuning and evaluate diverse receipt samples; continue receipt-intelligence accuracy testing.	Nov. 22	Model tuning/evaluation results	Evan & Jim
Week 13 Nov. 23-29	Prepare production configuration, deployment services, environment variables, migrations, and end-to-end release checklist.	Nov. 29	Deployment-ready build	All members
Week 14 Nov. 30-	Deploy application; validate web/mobile upload, fraud analysis, review, and export workflows in	Dec. 6	Hosted deployment	All members

Week / Dates	Primary Tasks / Activities	Tentative Deadline	Expected Checkpoint	Primary Owner(s)
Dec. 6	hosted environment.			
Week 15 Dec. 7-13	Implement/verify monitoring for false positives, processing time, database errors, and AI API costs; complete final evaluation and documentation.	Dec. 13	Monitoring + final evaluation	All members

8. Targeted Functionalities / Outcomes

- Improved receipt extraction pipeline with better accuracy on varied receipt layouts and image qualities.
- Scalable PostgreSQL data architecture with company-specific configuration, policy support, user authentication, and business-level data separation.
- Pattern-based anomaly detection integrated with the existing fraud-scoring pipeline.
- Enhanced web dashboard with statistics, risk scores, trends, and visualizations.
- Mobile receipt scanning/upload capability connected to the same analysis workflow as the web application.
- CSV and/or PDF review report export.
- Deployed application accessible in a production-style environment.
- Monitoring and evaluation results documenting accuracy, false positive rate, processing time, database health, and AI API cost.

9. Development Environment

Receiptly uses a web-service architecture with a Python backend, PostgreSQL data layer, React/TypeScript reviewer frontend, AI/ML services, and a planned mobile client. The development environment will continue to use Git-based version control and modular service layers so features can be developed and tested independently.

Layer	Technology	Purpose
Backend API	FastAPI, Python 3.11+	Asynchronous REST API and fraud-processing orchestration
Database	PostgreSQL, SQLAlchemy async ORM, Alembic	Receipt, company, user, policy, fraud flag, and review data
Receipt Extraction	Current Tesseract OCR; replacement/improved extraction library under evaluation; model-assisted fallback	Extract merchant, date/time, amount, and line items
Duplicate Detection	ImageHash / perceptual hashing (pHash)	Detect visually similar receipt submissions
AI Explanation	Gemini 2.5 Flash	Plain-English explanation of flagged receipts
Anomaly Detection	scikit-learn; planned pattern/history features	Behavioral and statistical anomaly signals
Web Frontend	React 18, TypeScript, Vite, Tailwind CSS, shadcn/ui	Reviewer dashboard, receipt list/detail, upload, review actions
Mobile	Mobile app/client to be developed in Fall 2026	Receipt scanning/photo capture and upload
Testing	pytest plus manual end-to-end testing	Regression and fraud-logic verification
Version Control	Git / GitHub	Feature-branch workflow and collaboration
Deployment	Render/Vercel/Supabase or equivalent hosted services	Production-style hosting and monitoring

10. Prototyping

The Spring 2026 MVP serves as the primary prototype for Fall 2026. Existing screens include a dashboard, receipt list, receipt detail view, and upload workflow. New CPSC 491 features will follow the same iterative approach: build a small working prototype, connect it to the existing API/data model, test with sample receipts, collect measurable results, and then refine the feature before full integration.

- Extraction prototype: compare alternative extraction libraries/engines on the same receipt samples before replacing the current path.
- Multi-business/authentication prototype: validate company-specific policy configuration, user access, and data separation with test organizations.
- Anomaly prototype: begin with interpretable historical patterns before expanding to additional model-based signals.
- Mobile prototype: implement receipt capture/upload first, then validate integration with the existing backend and reviewer workflow.
- Dashboard/export prototype: verify that metrics and exported reports match stored database results before visual polish.

11. Implementation Methods

11.1 Coding Standards

- Maintain the existing service-layer backend design so API routes, business logic, database models, and fraud detectors remain separated.
- Use versioned REST endpoints and Pydantic validation for backend request/response contracts.
- Use database migrations through Alembic rather than manual schema changes.
- Use feature branches and Git-based code review/merge practices to reduce conflicts between team members.
- Add or update automated tests when fraud-scoring, policy, duplicate, anomaly, or extraction logic changes.
- Keep reviewer-facing fraud results explainable by storing the specific signals/flags that contribute to each risk score.

11.2 AI Models and Detection Methods

- Receipt extraction: current Tesseract OCR with preprocessing; evaluate a replacement or enhanced extraction library/engine and use model-assisted fallback/validation for difficult receipts when appropriate.
- Duplicate detection: perceptual hashing (pHash) to identify visually similar receipt submissions.
- Policy detection: configurable rule-based validation using company-specific policy parameters.
- Risk scoring: weighted 0-100 score combining fraud signals, severity, and confidence.
- AI explanation: Gemini 2.5 Flash for concise reviewer-facing explanations of flagged receipts.
- Anomaly detection: scikit-learn-based methods and historical pattern detection, with tuning based on diverse receipt data and evaluation metrics.

12. Products

- Web-based Receiptly reviewer application.
- Mobile receipt scanning/upload client integrated with authenticated backend workflows.
- Backend REST API and PostgreSQL database.
- Fraud/anomaly detection and scoring pipeline.
- CSV/PDF review report exports.
- Deployed hosted application with monitoring and evaluation documentation.

13. Risks and Mitigations

Risk	Mitigation
Receipt extraction remains inconsistent for blurry, low-contrast, or unusual layouts.	Benchmark multiple extraction approaches, improve preprocessing, use low-confidence fallback/validation, and track extraction accuracy.
Anomaly detection creates too many false positives.	Tune thresholds on diverse test receipts, track false positive rate, preserve human review, and keep anomaly signals explainable.
Mobile integration takes longer than expected.	Prioritize capture/upload and backend connectivity first; defer nonessential mobile UI polish until the core workflow is stable.
Cloud/API usage exceeds free or expected cost	Cache AI results, monitor API usage/costs, rate-limit expensive operations,

limits.	and preserve local/open-source processing where practical.
Deployment introduces database or configuration failures.	Use migrations, environment-based configuration, staged deployment checks, backups, health endpoints, and database error monitoring.
Weekly schedule slips because model tuning or integration requires more iteration.	Use phased checkpoints, maintain a prioritized backlog, and preserve core deployment/monitoring deliverables before optional polish.

14. Team Member Contributions

Each team member has a defined primary area of responsibility for CPSC 491. The responsibilities below align the Fall 2026 implementation plan with individual ownership while allowing collaboration on integration, testing, deployment, and documentation.

Team Member	Primary CPSC 491 Contributions	Key Deliverables / Ownership
Jim Alvarez	Receipt intelligence and receipt-reading accuracy; CSV/PDF report extraction and export; shared dashboard development with Evan.	Improve extraction reliability across diverse receipts; implement review-report export; contribute dashboard metrics and presentation.
Evan Miller	Analytics; advanced fraud and anomaly detection; shared dashboard development with Jim.	Develop pattern-based/advanced fraud detection, tune fraud analytics, and build dashboard statistics, trends, scores, and visual graphs.
Syon Chau	Backend/frontend integration; multi-business operations; user authentication; mobile app receipt scanning and upload.	Connect frontend workflows to backend services, implement business-level operations and authenticated user access, and develop the mobile scanning/upload workflow.

Major integration, testing, deployment, and final documentation tasks will be completed collaboratively, while the feature areas below identify each member's primary contribution.

15. Conclusion

Receiptly enters CPSC 491 with a semi-functional Spring 2026 MVP and a clear path toward a more complete product. The Fall 2026 plan focuses on measurable receipt-reading improvement, scalable multi-business support, stronger anomaly detection, richer reviewer analytics, mobile receipt capture, exportable reports, deployment, and operational monitoring. The weekly project plan divides these goals into incremental checkpoints so the team can test and integrate each major feature before final deployment and evaluation in December 2026.